

Projektarbeit Machine Learning

Vergleich von Klassifizierungsalgorithmen

Muhammad Affandi Bin Mansor, 13.08.2026

Einführung

Einführung

Ausgangssituation

- Es sollte ein Vergleich von Klassifizierungsalgorithmen erstellt werden.
- Für den Vergleich wird ein Datensatz über Credit Scores analysiert.
- Der Credit Score ist hierbei in 3 Kategorien eingeteilt:
 - Low
 - Average
 - High

Einführung

Algorithmen

- Folgende Algorithmen stehen im Fokus dieses Projekts:
 - k-Nearest-Neighbors
 - Decision Tree
 - Neuronale Netze

Einführung

Projekt Repository

- Ein GitHub Repository wurde für dieses Projekt angelegt: [https://github.com/affandimansor/projektarbeit machine learning](https://github.com/affandimansor/projektarbeit_machine_learning)
- Alle relevanten Unterlagen (Quellcode, Datenmenge usw.) können aus diesem Repository lokal heruntergeladen werden.

Einführung

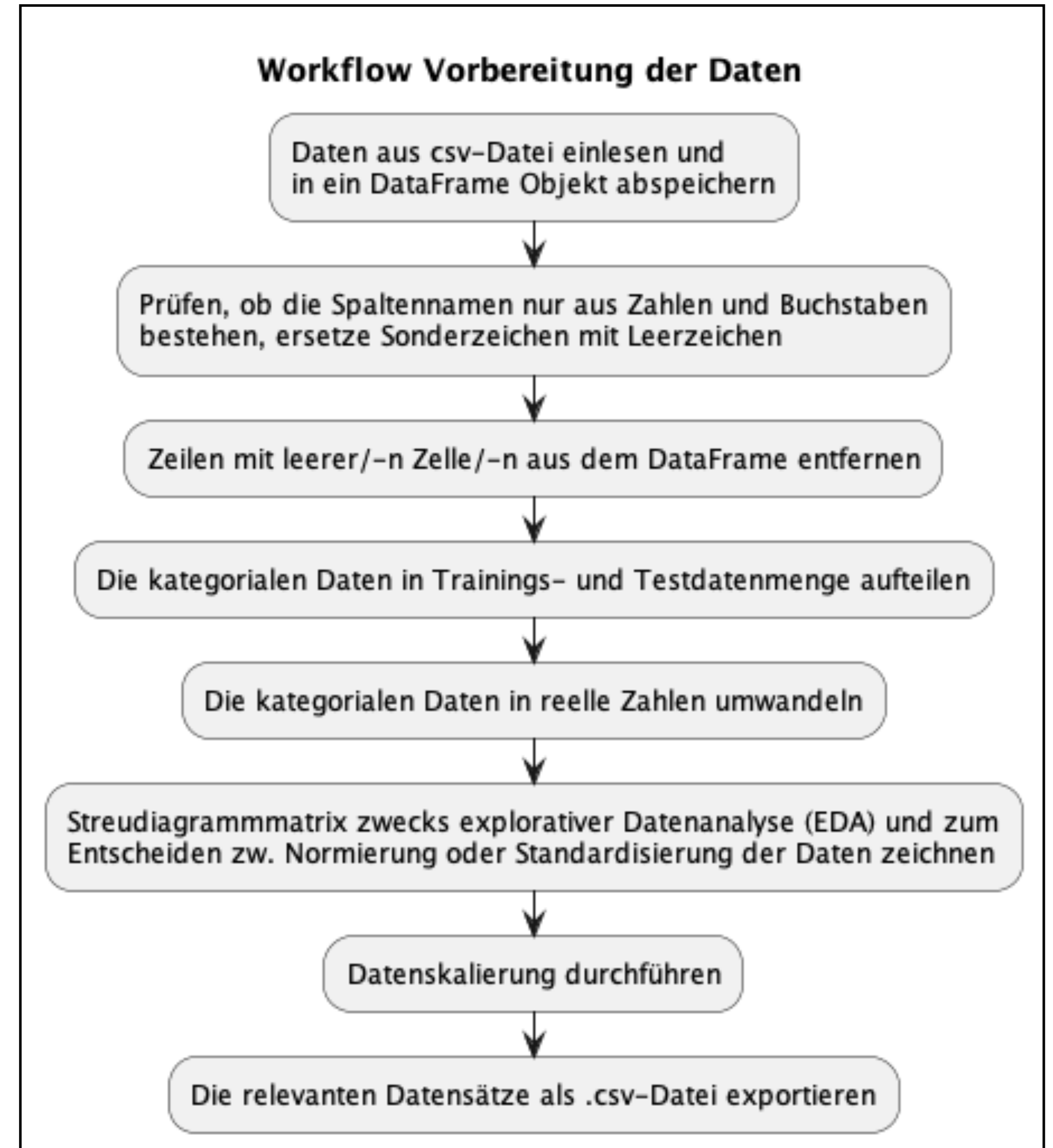
Darstellungsablauf der Ergebnisse jedes Algorithmus

- Die beste Parameterkombination.
- Confusion Matrix zur Darstellung der Vorhersage mit den Testdaten.
- Die Generalisierungseigenschaft zwecks Modellfitting. Dafür sind folgende zufällige Grenze (bzw. Threshold) eingesetzt:
 - Underfitting: 50%
 - Overfitting: 5%
- Schlussfolgerung

Analyseergebnis

Vorbereitung der Daten

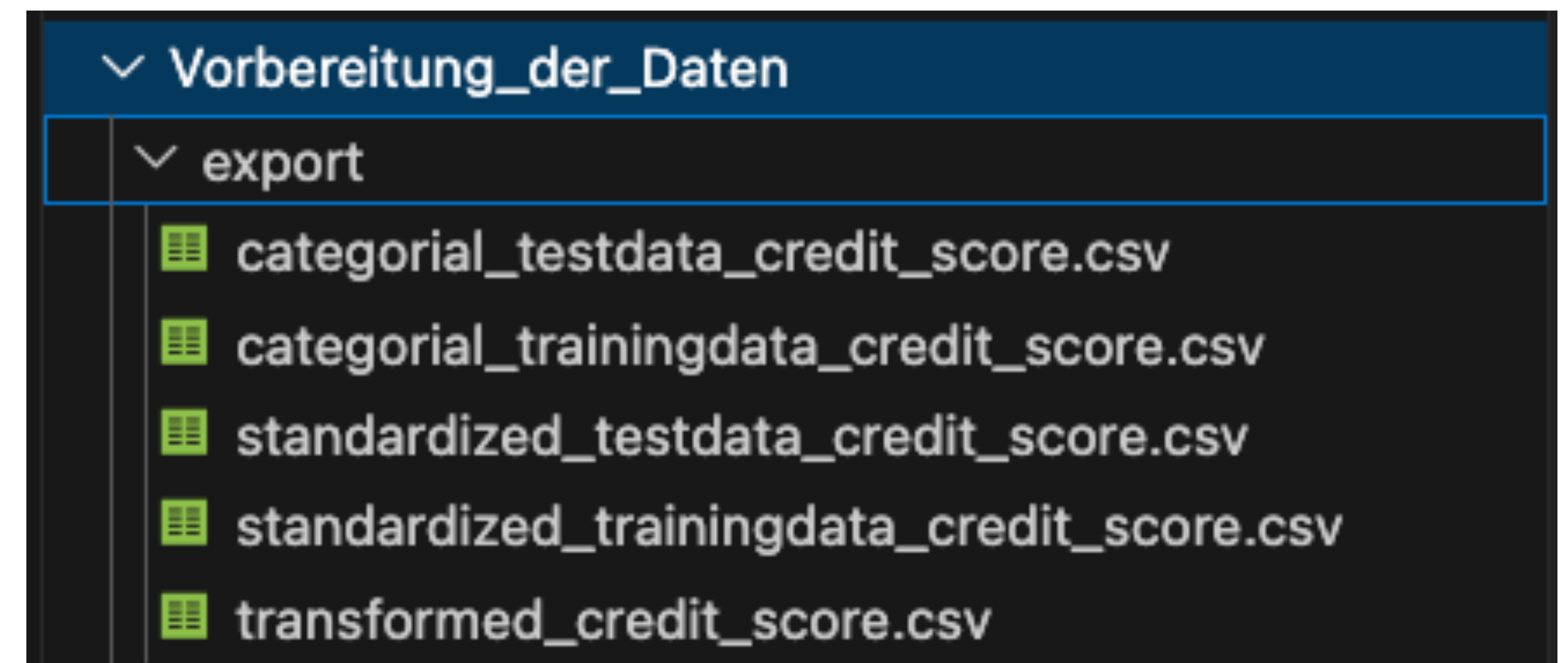
Workflow



Vorbereitung der Daten

Result

- Categorical: Die Daten im ursprünglichen Format.
- Transformed: Kategoriale Daten umgewandelt im numerischen Format durch die entsprechenden Encoder Klassen.
- Standardized: Standardisierte, numerische Daten.



Vorbereitung der Daten

Schlussfolgerung

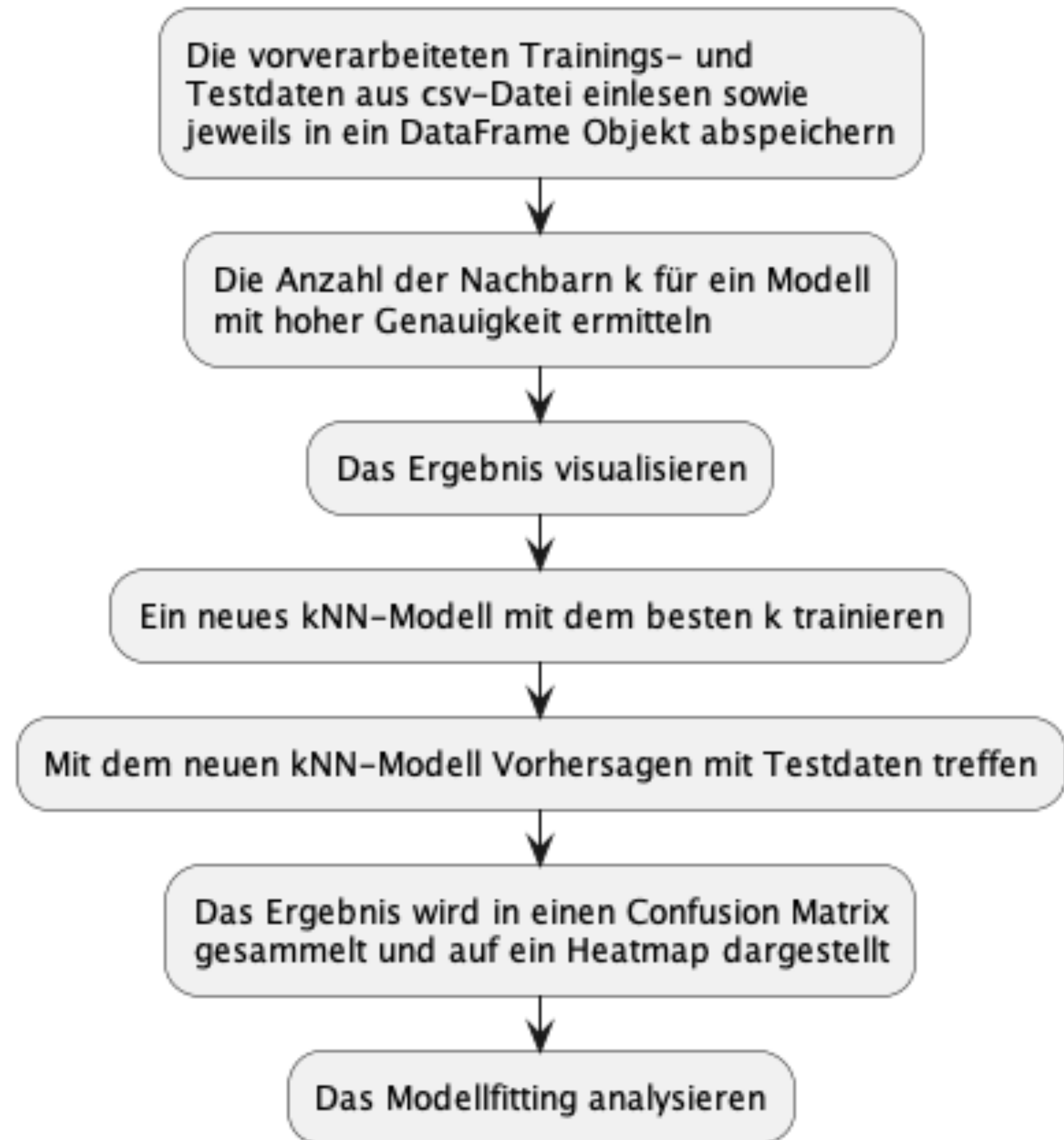
- Zwei unterschiedliche Encoder jeweils für nominale und ordinale Merkmale kamen zum Einsatz:
 - Ordinal: Education, Credit Score
 - Nominal: Alle anderen Merkmale
- Die Aufteilung der Datenmenge im Verhältnis 70%-Training und 30%-Test ergibt brauchbare Ergebnisse beim Analysieren des Datensatzes.
- Die standardisierten Datensätze sind zum Trainieren und Testen aller drei Algorithmen verwendet, da sie in der Pipeline bereits vorhanden sind.

k-Nearest-Neighbors

Workflow

- Der umgesetzte Workflow im Code zur Analyse des kNN Modells.

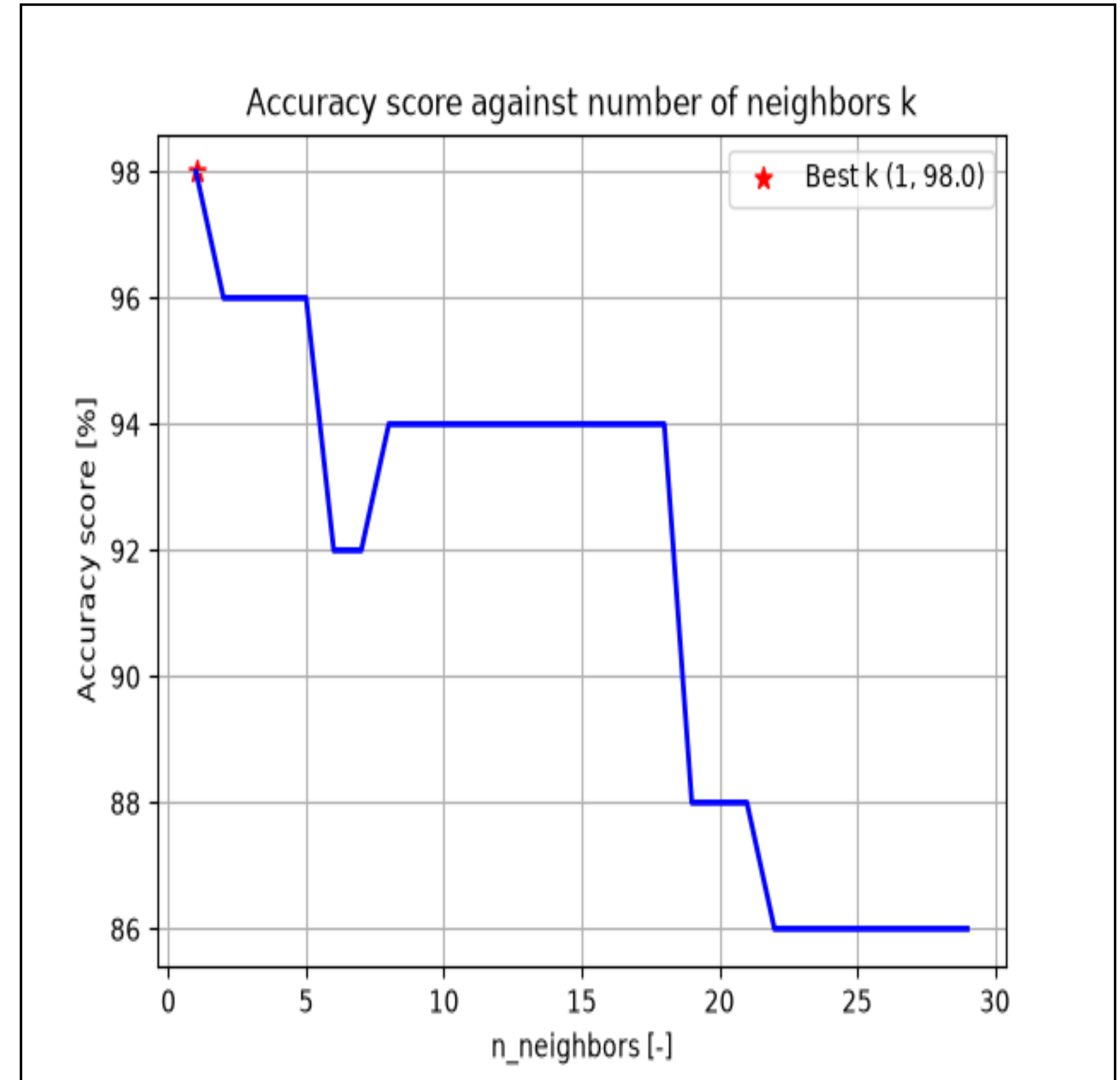
Workflow k-Nearest-Neighbors Analysis



k-Nearest-Neighbors

Result - Best k

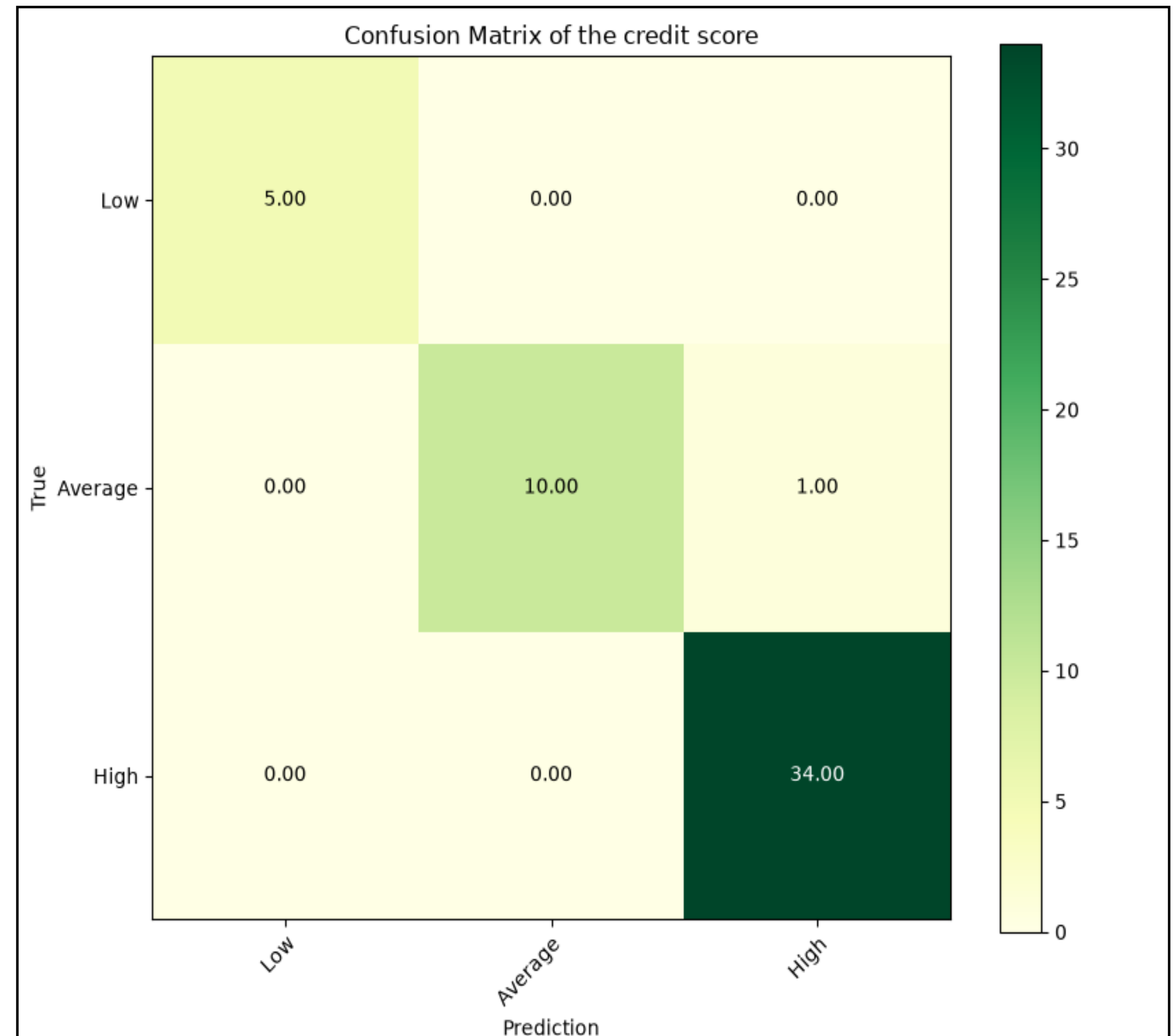
- Das kNN-Modell mit der Anzahl der Nachbarn $k = 1$ liefert den besten Accuracy Score von 98%.



k-Nearest-Neighbors

Result - Confusion Matrix

- Eine einzige falsche Vorhersage ist bei der Kategorie Average, welche diese als High klassifiziert ist.



k-Nearest-Neighbors

Result - Generalisierung

```
((.venv) ) affandimansor@MacBook-Air-von-Muhammad src % python k_nearest_neighbor_analysis.py  
accuracy_train: 98.0%; accuracy_test: 98.0%  
k-Nearest-Neighbors: Model generalizes well to unseen data; gap threshold = 5%
```

- Der gap threshold 5% stellt die maximale, zulässige Abweichung der Accuracy zw. Training und Test dar, bei der das Modell noch als „gut generalisiert“ bezeichnet werden kann.
- Das kNN-Modell zeigt sehr gute Generalisierungseigenschaft mit dem vorgegebenen Datensatz. Weiterhin signalisiert die konstante Accuracy zw. Training und Test ein optimales Modell.

k-Nearest-Neighbors

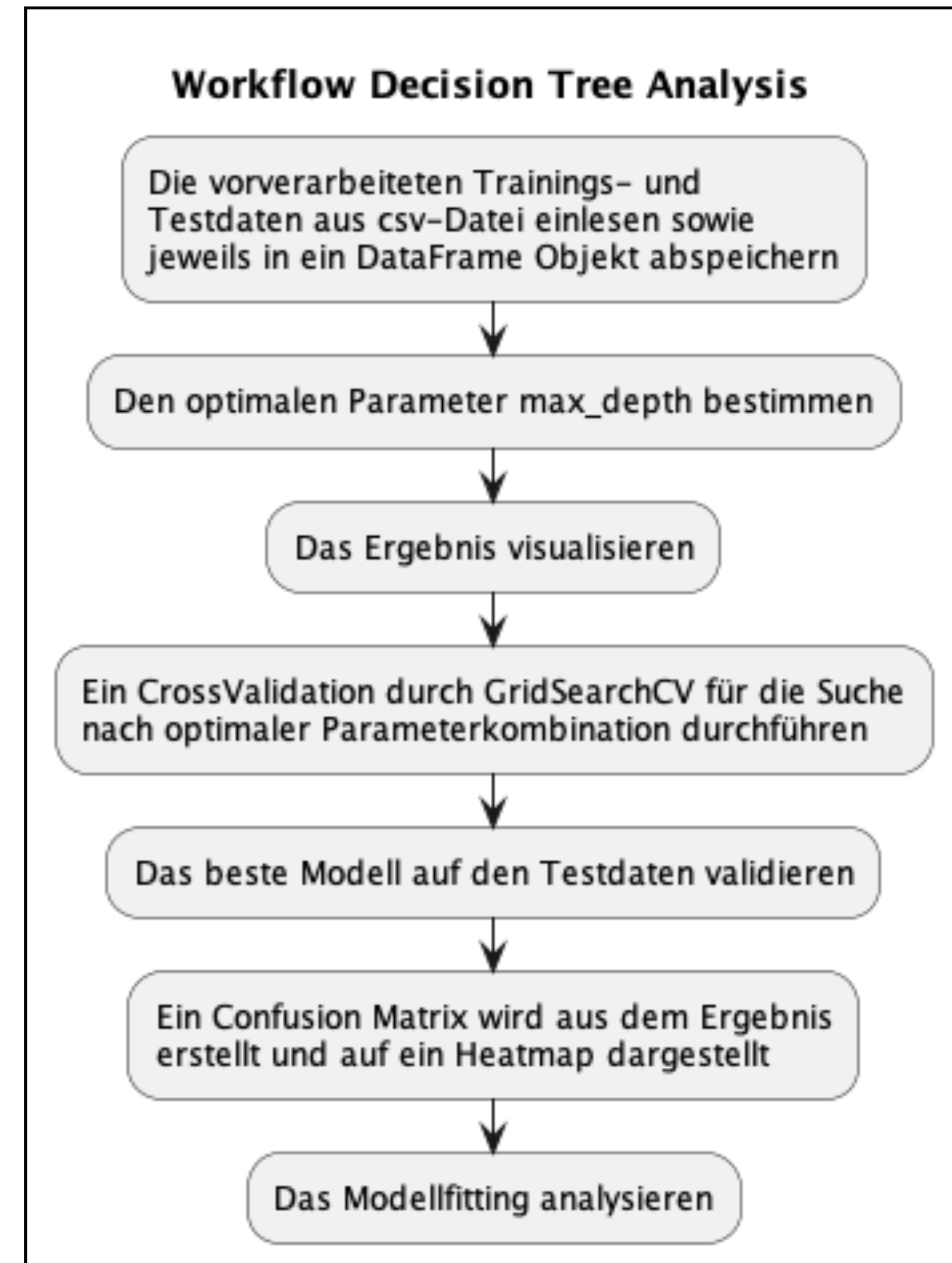
Schlussfolgerung

- Das kNN-Modell liefert einen hervorragenden accuracy_score von 98% und daher kann den vorgegebenen Datensatz gut beschreiben.
- Mit weiteren Datenpunkten vor allem für die Kategorie Low wird der accuracy_score vermutlich leicht gesunken.

Decision Tree

Workflow

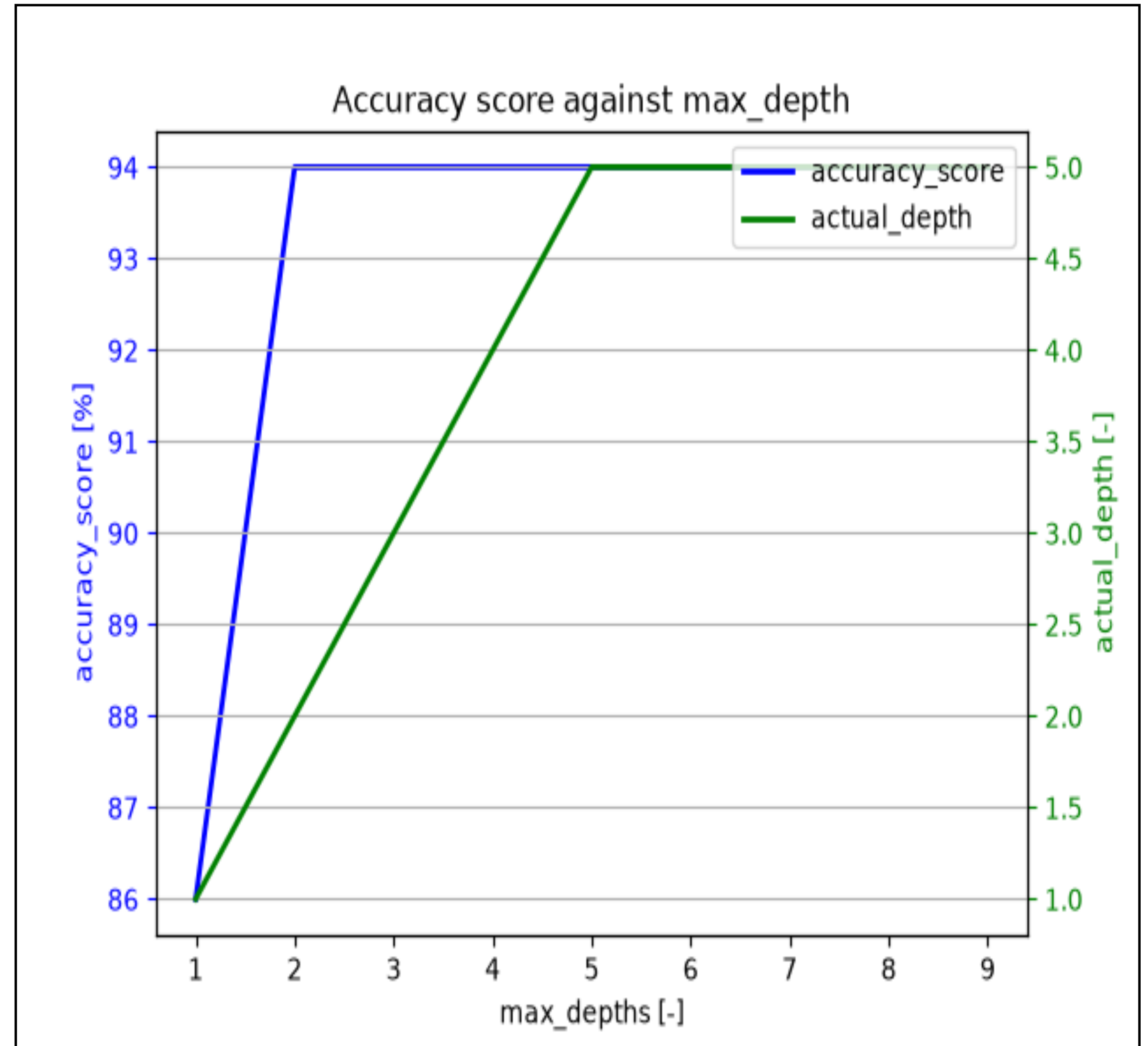
- Der umgesetzte Workflow im Code zur Analyse des Decision Tree Modells.
- Ein Hyperparameter tuning des Modells wurde durch CrossValidation mit GridSearchCV durchgeführt.



Decision Tree

Result - Best max_depth

- Die beste Accuracy liefert schon das Modell mit dem Parameter `max_depth = 2`.
- Das Modell erreicht seine maximale Tiefe bei `max_depth = 5`.



Decision Tree

Result - Hypertuning mit GridSearchCV

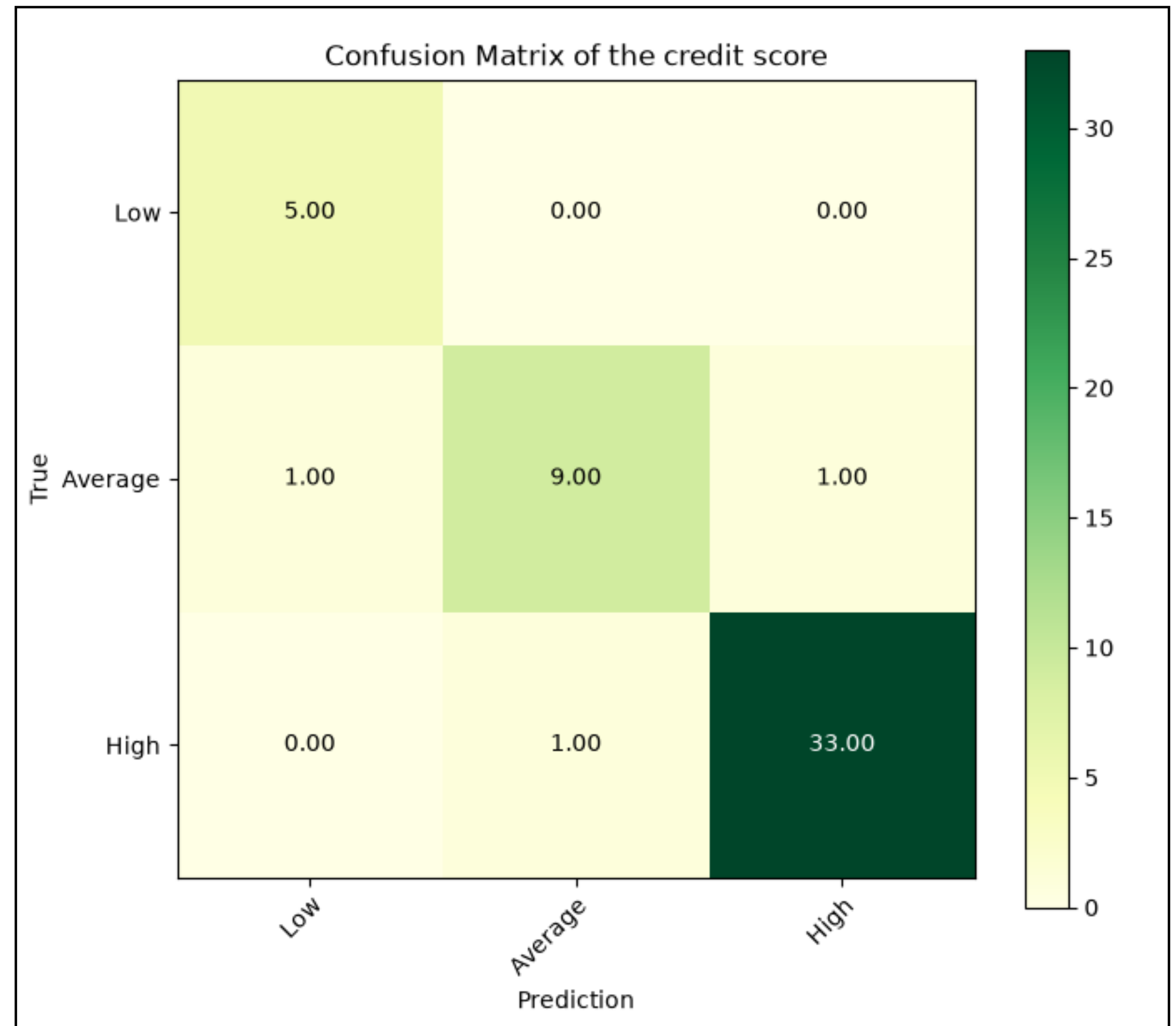
- GridSeachCV ermittelt und liefert folgende Parameterkonfigurationen eines optimalen Decision Tree Modell aus.
- Ein Decision Tree Modell mit diesen Parameterkonfigurationen wird erstellt und weiter mit den Testdaten validiert.

```
Best parameter combination:  
Parameter | Value  
-----  
ccp_alpha | 0.0  
criterion | gini  
max_depth | 3  
min_samples_leaf | 1
```

Decision Tree

Result - Confusion Matrix

- Das Modell kann einige Datenpunkte der Kategorien Average und High nicht richtig zuordnen.



Decision Tree

Result - Generalisierung

```
accuracy_train: 97.35%; accuracy_test: 94.0%  
Decision tree: Model generalizes well to unseen data; gap threshold = 5%
```

- Im Allgemein zeigt das Decision Tree Modell eine gute Generalisierung mit dem vorgegebenen Datensatz.

Decision Tree

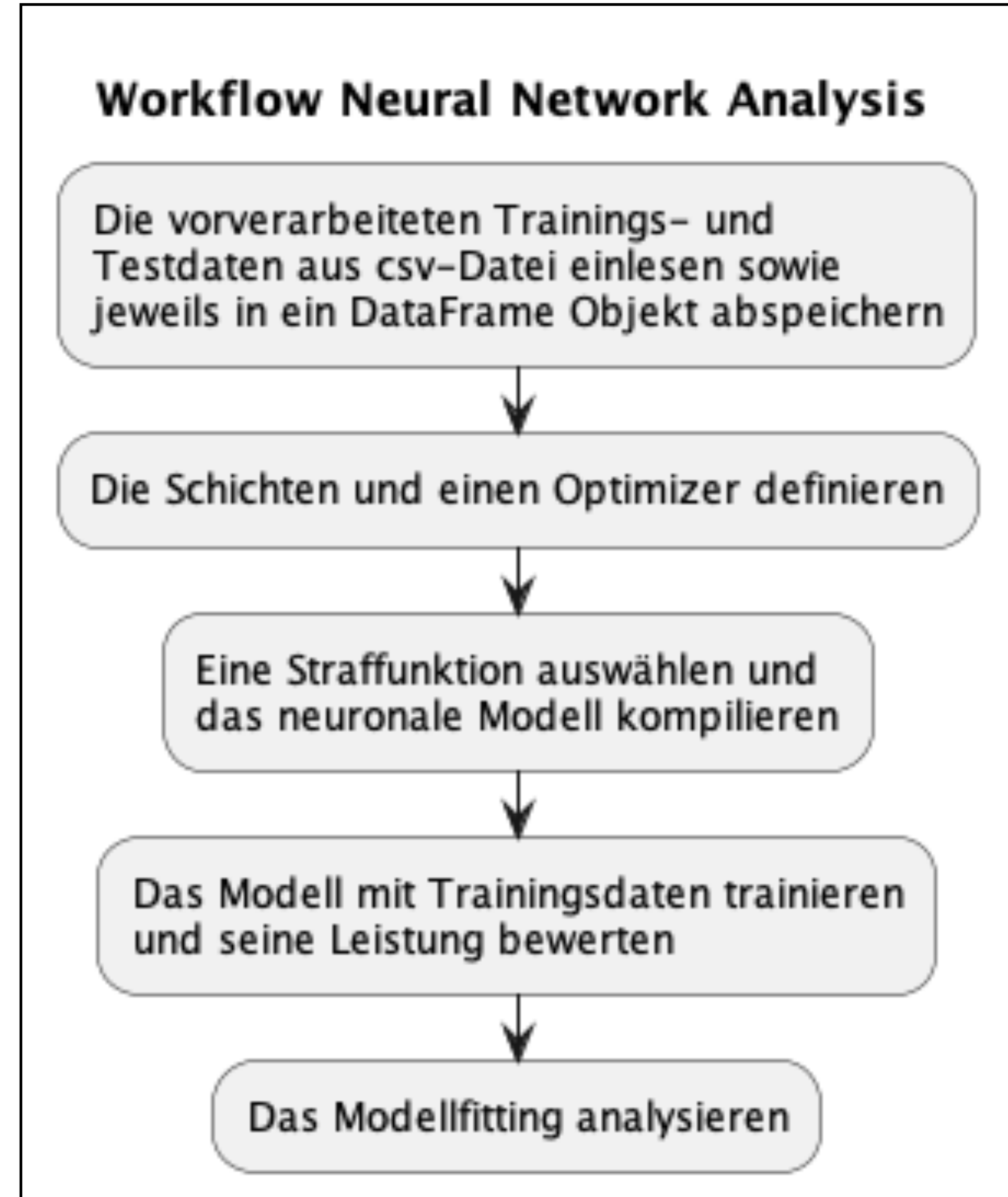
Schlussfolgerung

- Das kNN-Modell liefert einen guten accuracy_score von 97.35%. Daher kann es als ein Kandidat zum Repräsentieren des vorgegebenen Datensatzes berücksichtigt werden.
- Mit weiteren Datenpunkten vor allem für die Kategorie Low wird einerseits der accuracy_score vermutlich leicht gesunken. Andererseits kann die Accuracy des Modells zum Vorhersagen der Kategorie Average und High durch das Hinzufügen weiterer Datenpunkte verbessert werden.

Neuronales Netzwerk

Workflow

- Der umgesetzte Workflow im Code zum Erstellen und Bewerten eines einfachen neuronalen Netzwerk Modell.



Neuronales Netzwerk

Result - Generalisierung

```
accuracy_train: 15.384615957736969%; accuracy_test: 14.000000059604645%  
Neural Networks: Model is underfit; underfit threshold = 50%
```

- Das einfache neuronale Netzwerk Modell ist deutlich underfit und ist deshalb nicht in der Lage den vorgegebenen Datensatz zu beschreiben.
- Weiterhin kann kein Confusion Matrix mit der aktuellen Konfiguration des Modells erstellt werden. Die Modell Konfiguration ist auf dem Remote Repository in dem folgenden Script zu finden: [neural_networks_analysis.py](#).

Neuronales Netzwerk

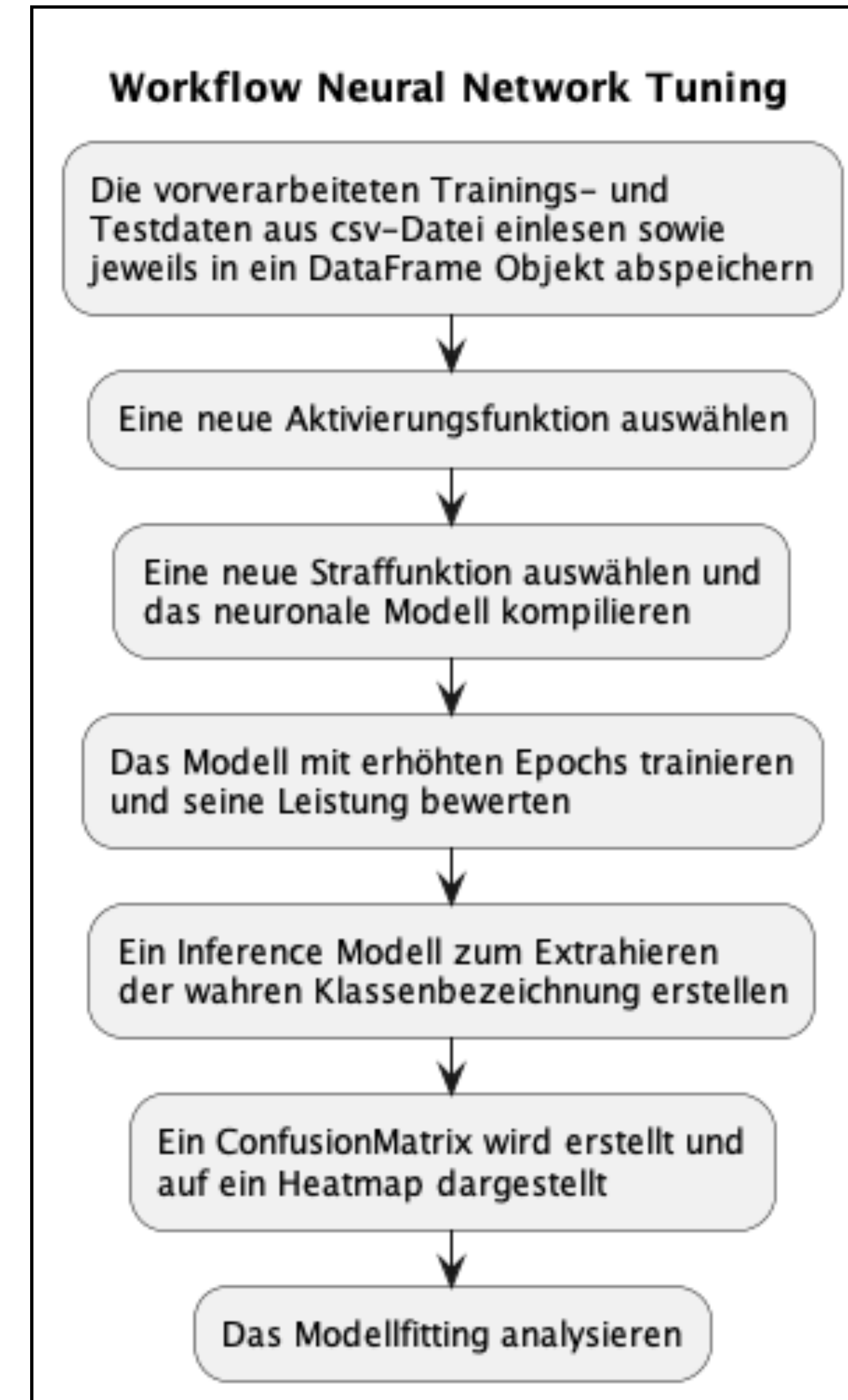
Schlussfolgerung

- Das Modell muss unbedingt neu konfiguriert werden, um den Modelfit zu verbessern und weiterhin eine Confusion Matrix erstellen zu können.
- Im nächsten Abschnitt ist das optimierte neuronale Netzwerk Modell beschrieben.

Neuronales Netzwerk Tuning

Workflow

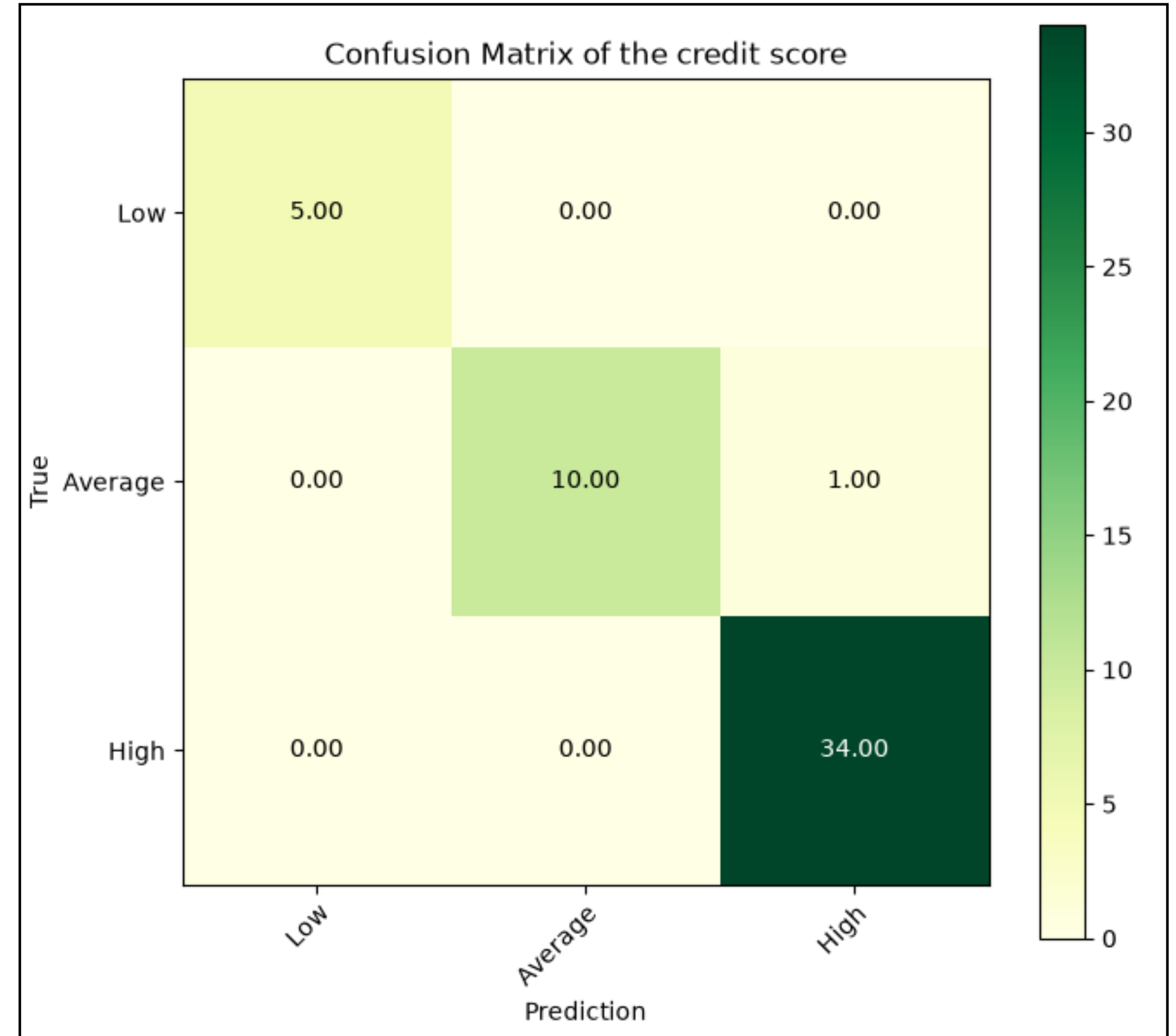
- Das vorhin erwähnte neuronale Netzwerk Modell wird gemäß dem dargestellten Workflow optimiert.



Neuronales NetzwerkTuning

Result - Confusion Matrix

- Mit den Testdaten zeigt das optimierte neuronales Netzwerk Modell das gleiche Ergebnis wie bei dem kNN-Modell.
- Eine einzige falsche Vorhersage ist wieder bei der Kategorie Average, welche diese ebenfalls als High klassifiziert ist.



Neuronales Netzwerk Tuning

Result - Generalisierung

```
accuracy_train: 100.0%; accuracy_test: 98.00000190734863%  
Optimized Neural Networks: Model generalizes well to unseen data; overfit threshold = 5%
```

- Das optimierte neuronale Netzwerk Modell liefert einen ausgezeichneten accuracy_score von 100% während des Trainings.
- Dasselbe Modell weicht aber von den Testdaten leicht ab. Nichtsdestotrotz ist der accuracy_score von 98% während des Tests ein gutes Ergebnis.
- Somit weist das Modell eine gute Generalisierungseigenschaft auf.

Neuronales Netzwerk Tuning

Schlussfolgerung

- Das erhöhte Epochs von 50 auf 100 hat die Accuracy verbessert.
- Die Straffunktion Sparse Categorical Crossentropy ermöglichte das Erstellen einer Confusion Matrix und trug ebenfalls zur Verbesserung der Accuracy bei.
- Die Accuracy Abweichung zw. Training und Test lässt sich vornehmlich durch erweiterte Datenpunkte reduzieren.

Abschluss

Zusammenfassung

- Alle drei Algorithmen weisen fast identische Accuracy beim Vorhersagen anhand der Testdaten auf.
- Das kNN-Modell kann einfachheitshalber als bevorzugtes Modell zum Beschreiben des vorgegebenen Credit Score Datensatzes eingesetzt werden.
- Falls bei einem Datensatz die Leistungsgrenze des kNN-Modelles überschritten ist, kann das mächtigere, neuronale Netzwerk Modell auf ihm angewendet werden.